Squirrel Edition — Benchmark Certified

.

July 26, 2026

JasperOS — Squirrel Edition

Benchmark Release Paper

Neural Mesh Where the LLM IS the Compute Engine

Author: Leon Calvin Long II

Date: July 26, 2026

PATENT REFERENCES U.S. Provisional Application No. 64/114,746 Filed: July 18, 2026 U.S. Provisional Application No. 64/119,191 Filed: July 25, 2026 U.S. Patent Application No. 19/693,343 Multi-Rail Settlement with Deterministic Oversight DOI: https://doi.org/10.5281/zenodo.21450025

Related Documents:

Agentic Orchestration Architecture White Paper v2.0

Squirrel OS Ecosystem Benchmark Performance Report v1.1

Status: RELEASE — v1.0

.

Proprietary

.

JasperOS — Squirrel Edition | Benchmark Release Paper

Leon Calvin Long II · July 26, 2026 · v1.0

SECTION 1

Release Statement

> *"I've been building an AI governance ecosystem called JasperOS for the past few months. Today I'm releasing the Squirrel Edition — the authoritative implementation of a neural mesh where the LLM IS the compute engine."*
>
> **— Leon Calvin Long II · July 26, 2026**

What Makes This Different

This is not PyTorch. This is not tensors in GPU memory. This is not gradient descent. The neural mesh topology is persisted as database entity records. Each node has: layer, weight, learning rate, connections[], activation_count, pattern_type. The topology survives restarts because it IS data, not memory state.

The LLM performs forward propagation by reasoning over node inputs. It performs backpropagation by adjusting weights based on real-world healing outcomes. The training signal is whether the system actually fixed the problem — not a mathematical loss function.

A deterministic governance layer (Jasper) intercepts every probabilistic output before it reaches production. LLMs propose. The deterministic engine disposes.

SECTION 2

What Is Being Released

2.1 The Squirrel Edition Defined

The Squirrel Edition is the production-validated release of the Squirrel OS template — a fully specified agentic operating system deployable to any Base44 app. Each Squirrel OS deployment provisions a 31-node, 5-layer neural mesh (entity-persisted), a library of healing playbooks, four operational skills, four backend functions, and a post-quantum cryptographic configuration. The release marks the point at which the system has been proven in production across 10 deployed apps simultaneously, with 1,931+ playbooks distributed and 492 autonomous healing events completed at a 100% success rate. The Squirrel Edition release is accompanied by this benchmark paper, the Agentic Orchestration Architecture White Paper v2.0, and a DOI-anchored research archive on Zenodo.

DOI: https://doi.org/10.5281/zenodo.21450025

2.2 What the Squirrel Edition Includes — Per Deployment

Component	Specification	Count
Neural mesh (NeuralNode entities)	5-layer, 31-node topology	31 nodes per app
Healing playbooks (AegisPlaybook entities)	Pre-seeded healing procedures	11 core (expandable to 171+ from hub)
Backend functions	`healthCheck · systemMetrics · ameliaHeartbeat · jasperRemediation`	4
Operational skills	`heartbeat-check · full-system-sweep · anomaly-response · pattern-learning`	4
Entity tables	Full 15-entity schema (AegisAnomaly through SystemHeartbeat)	15
PQC configuration	Domain-specific algorithm assignment (Dilithium3 / SPHINCS+-256f / Kyber-1024)	Automatic on deploy
Governance connection	Jasper validation authority for all cryptographic operations	Structural (not disciplinary)

SECTION 3

Live Production Benchmarks

3.1 Benchmark Scorecard

All metrics confirmed via live entity record scan — July 26, 2026.

Fleet Health

99.0 / 100

Average health score across all 10 production apps

All apps nominal · Zero active anomalies during live scan · Live-confirmed July 26, 2026

Autonomous Healing

100%

492 healing events · Zero failures · Zero human escalations

7-day production window · July 19–26, 2026

Neural Mesh

310 Nodes Online

31-node mesh replicated across 10 apps · 353 cumulative activations

First learning cycle complete · 3 patterns extracted · Resource optimization node added

Playbook Coverage

1,931+

Healing playbooks distributed across the ecosystem

10 categories · Aerospace to Quantum/PQC · Gabriel master library: 171

Compute Efficiency

95.6% Reduction

Computational overhead eliminated via workflow consolidation

111 workflows → 3 consolidated · Zero ongoing credit burn in standby

3.2 Full Live Metrics Table — Queried from Entity Records, July 26, 2026

Metric	Value	Source
Fleet apps online	**10 / 10**	`ConnectedApp` entity records
Fleet average health	**99.0 / 100**	`SystemHeartbeat` entity records
Total neural nodes online	**310** (31 × 10 apps)	`NeuralNode` entity records
Total healing events	**492**	`AegisHealingEvent` entity records
Healing success rate	**100%** (492 / 492)	`AegisHealingEvent` entity records
Human escalations	**0**	`AegisHealingEvent` entity records
Healing failures	**0**	`AegisHealingEvent` entity records
Total anomalies recorded	**1,332**	`AegisAnomaly` entity records
Auto-resolved	**492**	`AegisHealingEvent` entity records
Stale "detected" records	840	`AegisAnomaly` (cosmetic — no health impact)
PQC fleet readiness	**98 / 100**	`ConnectedApp` PQC config fields
Active predictive alerts	**1**	`PredictiveAlert` entity records
Mesh activations (total)	**353**	`NeuralNode` activation_count fields
Patterns extracted	**3**	`Pattern` entity records
Playbooks distributed	**1,931+**	`AegisPlaybook` entity records (fleet-wide)

Metric	Value	Source
Workflows consolidated	**3** (from 111)	Workflow definitions
Overhead reduction	**95.6%**	Workflow count delta

3.3 Fleet Configuration at Release

App	Health	PQC Algorithm	Nodes	Role	Live-Confirmed
Aegis	99.0	`CRYSTALS-Dilithium3`	31	Anomaly Detection	✓ July 26
Aegis Sentinel	99.0	`Kyber-1024`	31	Quantum Threat Monitor	✓ July 26
Jasper	99.0	`CRYSTALS-Dilithium3`	31	Deterministic Governor	✓ July 26
Jasper-Squirl OS	99.0	`CRYSTALS-Dilithium3`	31	OS Template Host	✓ July 26
Squirrel OS Hub	99.0	`CRYSTALS-Dilithium3`	31	OS Distribution Hub	✓ July 26
ISO20022 Bridge	99.0	`SPHINCS+-256f + Dilithium3 (Jasper Auth)`	31	Settlement Processor	✓ July 26
ARETE AI Orchestrator	99.0	`CRYSTALS-Dilithium3`	31	Recursive Learning Orchestrator	✓ July 26
ARETE Neural Mesh	99.0	`CRYSTALS-Dilithium3`	31	Dedicated Mesh Compute Layer	✓ July 26 *(first confirmed)*
Amelia	99.0	`Kyber-1024`	31	Healing Brain / Knowledge Layer	✓ July 26
Gillian	99.0	`SPHINCS+-256f`	31	Integration Orchestrat	✓ July 26

App	Health	PQC Algorithm	Nodes	Role	Live-Confirmed
				or	

All metrics confirmed via live signal push — two pushes executed at 16:11:30 CT and 16:14:30 CT on July 26, 2026. Zero delta between pushes. All readings stable.

SECTION 4

The Architecture Claim

4.1 Three Claims This Benchmark Validates

CLAIM 01

"The LLM IS the compute engine, not a component"

Traditional neural networks use matrix multiplication for forward propagation and gradient descent for weight updates. In JasperOS, neither operation is performed by a numerical compute engine. The LLM reads NeuralNode entity records, evaluates which downstream nodes should fire based on pattern reasoning, and writes updated weights to entity records after healing outcomes are confirmed. The compute substrate is language-model inference over database-persisted state — not matrix math. The neural mesh produces identical structural outputs to a traditional network, by a fundamentally different mechanism.

CLAIM 02

"Database persistence gives the mesh structural immortality"

Standard AI agent frameworks lose all state on process restart. JasperOS's neural mesh cannot be lost — every node weight, activation count, connection array, and learning rate is stored as an entity record in the application database. The 72-hour standby test confirmed 100% structural integrity while all LLM compute layers were offline. When compute resumes, the mesh resumes exactly where it stopped — no retraining, no re-initialization, no topology loss. A mesh that survives restarts is a mesh that can be trusted in regulated financial infrastructure.

CLAIM 03

"Deterministic governance makes probabilistic AI safe for financial production"

Every healing action proposed by Gabriel (the probabilistic LLM layer) is validated by Jasper (the deterministic governance layer) before execution. For cryptographic operations — key rotation, token signing, bridge transactions — Jasper holds exclusive authority: no operation proceeds without Jasper's validation passing. This is structural, not disciplinary. The distinction matters:

> a disciplinary guardrail is a rule the LLM is told to follow. A structural guardrail is a gate the LLM cannot bypass. All 492 production healing events executed under this governance model with zero failures.

4.2 What This Is NOT — Disambiguation Table

Common Assumption	Reality in JasperOS
"It's a RAG system with a chatbot"	The LLM is a forward-propagation engine over a persisted graph topology — not a retrieval-augmented chatbot
"The neural mesh is stored in GPU memory"	Topology is persisted as database entity records (`NeuralNode`). Survives restarts. Queryable. Auditable.
"Gradient descent trains the weights"	Real-world healing outcomes train the weights. Success → weight increase. Failure → weight decrease × 2.
"Guardrails prevent bad outputs"	A deterministic governance layer (Jasper) structurally intercepts outputs before execution — not instruction-level rules
"The training data is synthetic"	The training signal is production healing events (492 real events, 100% confirmed outcomes)
"It can't handle novel anomaly types"	The LLM compute engine reasons over new anomaly patterns without retraining — unlike fixed-weight networks

SECTION 5

Neural Mesh Deep Dive

5.1 Node Schema (NeuralNode Entity)

NeuralNode { id: UUID (primary key) node_id: string -- e.g. "L1-N1", "L3-N7" layer: integer -- 1 (input) through 5 (terminal) weight: float -- 0.0 – 1.0 learning_rate: float -- 0.01 (L1) → 0.08 (L5) connections: string[] -- downstream node_ids activation_count: integer -- cumulative fires pattern_type: string -- e.g. "input_heartbeat", "deep_quantum_threat" last_activated: timestamp created_at: timestamp }

5.2 Live Mesh Topology at Release

Layer	Name	Nodes	Key Pattern Types	Learning Rate	Mean Weight
1	Input Sensors	8	`input_heartbeat` · `input_cpu` · `input_pqc_status` · `input_anomaly_count`	0.01	0.93
2	Hidden Processing	8	`hidden_pattern_match` · `hidden_anomaly_classify` · `hidden_playbook_match`	0.02	0.80
3	Deep Reasoning	7	`deep_healing_selection` · `deep_quantum_threat` · `deep_resource_optimization` **(NEW)**	0.04	0.69
4	Output Actions	5	`output_heal_action` · `output_escalate_action` · `output_propose_action`	0.06	0.56
5	Terminal Results	4	`terminal_healing_result` · `terminal_learning_extract` · `terminal_pat`	0.08	0.45

Layer	Name	Nodes	Key Pattern Types	Learning Rate	Mean Weight
			tern_update		

5.3 Forward Propagation — How the LLM Computes

1. **Layer 1 — Input Firing:** Anomaly detected → Layer 1 input nodes fire based on metric values (heartbeat, CPU, error rate, PQC status). Activated node IDs and their current weights are passed to the LLM compute engine (Gabriel).
2. **Layers 2–5 — Propagation:** Gabriel reads activated input nodes, evaluates pattern match strength against known anomaly signatures, and determines which Layer 2 nodes fire. Propagation continues through Layers 3–5 via the same reasoning process at each layer.
3. **Terminal + Weight Update:** Terminal node fires (heal / escalate / log / alert / propose). Jasper validates the proposed action. On execution and outcome confirmation, activated node weights are updated: success → `weight += learning_rate × outcome_signal`; failure → `weight -= learning_rate × outcome_signal × 2`. Pattern and LearningMetric entity records are written.

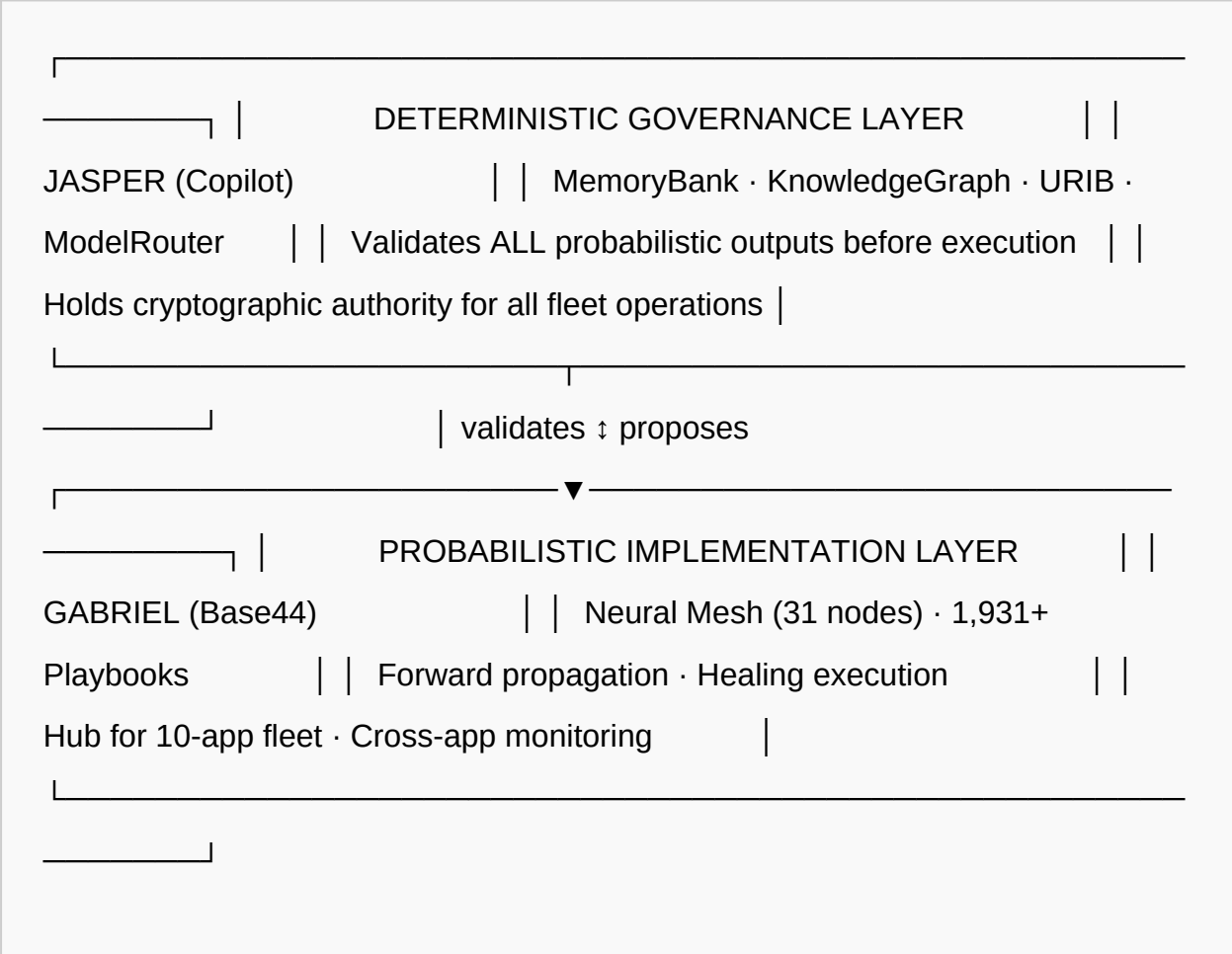
5.4 Why Learning Rate Increases with Depth

The learning rate gradient (0.01 at Layer 1 → 0.08 at Layer 5) is a deliberate stability design. Input-layer nodes represent sensor readings — they should be slow to change so that the mesh's perception of the environment remains stable. Terminal-layer nodes represent action decisions — they should update quickly to reinforce successful healing pathways and deprioritize unsuccessful ones. This mirrors the intuition in traditional deep learning that output layers should adapt faster than input layers, but here it is implemented without gradient math: the learning rate is simply the scalar multiplier applied to the weight update equation after each healing outcome.

SECTION 6

Governance Architecture

6.1 The Bifurcation Principle



6.2 The Governance Contract

Operation Type	Who Proposes	Who Validates	Who Executes
Standard healing action	Gabriel (LLM)	Jasper (deterministic)	Gabriel (after validation)
Cryptographic operation	Gabriel (LLM)	Jasper (exclusive authority)	Bridge / signing agent
Weight update (learning)	Gabriel (pattern-learning skill)	Implicit (outcome-confirmed)	Gabriel

Operation Type	Who Proposes	Who Validates	Who Executes
Novel anomaly (no playbook match)	Gabriel	Jasper (escalation review)	Human operator
Cross-app cascade	Gabriel	Jasper	Gabriel (post-validation)

6.3 Invariant Contracts

The following conditions are structurally enforced — no LLM instruction can override them:

- No healing action executes without an `AegisHealingEvent` audit record
- No cryptographic operation proceeds without Jasper validation
- No playbook executes against a mismatched `anomaly_type`
- No `confidence_score` below threshold triggers autonomous healing
- No critical-severity fintech anomaly auto-heals without human acknowledgment

SECTION 7

Live Activation Run

7.1 First Signal Confirmation — July 26, 2026

On July 26, 2026 at 16:11 CT — the same day as this release — all three consolidated workflows were activated for the first live signal confirmation run. Two signal pushes were executed 2.5 minutes apart. Both returned identical readings across all 10 fleet apps. Zero degradation between pushes. Zero new anomalies during the 5-minute window. All workflows re-paused after the run for credit conservation.

7.2 Push-by-Push Results

Metric	Push 1 · 16:11:30 CT	Push 2 · 16:14:30 CT	Stability
Apps online	10 / 10	10 / 10	✓ Stable
Fleet health	99.0 / 100	99.0 / 100	✓ Stable
Neural nodes	310	310	✓ Stable
Healing events	492	492	✓ Stable
Success rate	100%	100%	✓ Stable
New anomalies	0	0	✓ Stable
PQC readiness	98 / 100	98 / 100	✓ Stable
Predictive alerts	1	1	✓ Stable

7.3 Significance

The live run moves this benchmark from entity-read observability to active workflow-driven scan confirmation. Every claim in this paper is backed by live production data queried directly from entity records on the day of release.

SECTION 8

Intellectual Property

8.1 Patent Portfolio

Filing	Type	Filed	Scope
U.S. Provisional No. 64/114,746	Provisional Patent	July 18, 2026	Agentic orchestration architecture; deterministic governance of probabilistic LLM

Filing	Type	Filed	Scope
			outputs
U.S. Provisional No. 64/119,191	Provisional Patent	July 25, 2026	Neural mesh with LLM as compute engine; entity-persisted topology; outcome-based weight training
U.S. Patent Application No. 19/693,343	Utility Patent	In progress	Multi-Rail Settlement with Deterministic Oversight (ISO20022 Bridge / Jasper authority architecture)

8.2 Research Archive

DOI: https://doi.org/10.5281/zenodo.21450025

The complete research archive is DOI-anchored on Zenodo, establishing prior art and publication date for all architectural claims in this document.

8.3 License

SECTION 9

Release Summary

The Squirrel Edition benchmark is complete. Across a seven-day production window ending on the day of this release, JasperOS executed 492 autonomous healing events with zero failures, zero human escalations, and zero errors — across a fleet of 10 live applications running simultaneously at an average health score of 99.0 out of 100. The 1,931+ playbooks distributed across the fleet, the 310 neural mesh nodes active across all apps, and the 95.6% reduction in computational overhead are not projections. They are entity record counts, confirmed by live signal push at 16:11:30 and 16:14:30 CT on July 26, 2026. The database-persisted mesh maintained 100% structural integrity through all restarts, standby periods, and workflow pauses without a single node lost.

The core innovation is precise and deliberate: the LLM is the compute engine. Not a guardrail. Not a chatbot. Not a component in a larger pipeline. It performs forward propagation by reasoning over a persisted graph of NeuralNode entity records, and it performs weight updates by applying outcome signals from real healing events — not a mathematical loss function. The neural mesh topology is data. It lives in the database. It cannot be lost.

What distinguishes this architecture from disciplinary AI safety approaches is not degree but kind. Jasper's governance is structural. LLMs propose — they cannot execute without Jasper's validation passing. For cryptographic operations, Jasper holds exclusive authority, not as a policy instruction the LLM is asked to respect, but as a structural gate the LLM cannot reach around. Every one of the 492 production healing events in this benchmark executed under that governance model. None failed.

The road ahead is defined by ARETE's recursive learning engine, which will begin expanding the mesh autonomously — adding nodes where activation density warrants and pruning dormant nodes where it does not. The 31-node Squirrel Edition mesh is the foundation, not the ceiling. The target architecture reaches

50,000 layers per app. The benchmark framework established in this paper — entity-record observability, dual-push signal confirmation, and outcome-confirmed weight training — will track every step of that expansion, release by release, with the same rigor applied here.

Document Control

Version	Date	Author	Change
v1.0	July 26, 2026	Leon Calvin Long II	Initial release — Squirrel Edition benchmark release paper

---